Cost-Effective Numerical Question Answering over Semi-Structured Table

Feng Luo[1], Hui Luo[2], Zhifeng Bao[3], J. Shane Culpepper[3], Xiaoli Wang[4]
[1]RMIT University, [2]University of Wollongong, [3]The University of Queensland, [4]Xiamen University

ABSTRACT

Semi-structured tables encode rich semantic information through diverse layout elements, such as hierarchical row and column headers. Answering numerical questions over such data is challenging because it requires a joint effort of accurate structural understanding of tables, question uncertainty resolution, and complex query intents interpretation. Existing methods are rarely effective in handling the above multifaceted challenges in a holistic manner; moreover, they rely on LLMs without considering financial implications.

We propose SemiBonsai, a cost-effective, LLM-powered framework featuring: (1) a Multiway Layered Table Structurer that converts a table into a multiway layered tree organized by structural elements; (2) a Context-Aware Uncertainty Resolver that grounds underspecified phrases to context-coherent structural elements; (3) a Plan–Guided Reasoner that decomposes complex query intents into a query plan for reasoning; (4) a Budget-Aware Bandit Router that learns per-instance LLM utilities and optimizes LLM selection under fixed monetary constraints. Experiments on three benchmarks show that SemiBonsai improves both QA effectiveness and cost-effectiveness. Notably, the Table Structurer improves answer accuracy by 46%, and Router achieves a 5% average relative gain in answer accuracy compared with existing methods across budgets.

PVLDB Reference Format:
Feng Luo[1], Hui Luo[2], Zhifeng Bao[3], J. Shane Culpepper[3], Xiaoli Wang[4]. Cost-Effective Numerical Question Answering over Semi-Structured Table. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/JrJessyLuo/SemiBonsai.

1 INTRODUCTION

Semi-structured tables flexibly capture complex layouts common in real-world data (e.g., financial reports [39] and electronic health records [16]). Unlike flat tables, semantics can be (implicitly) encoded through rich layout elements, including the use of multiple subtables, hierarchical row headers, and hierarchical column headers [45]. Notably, a substantial portion of the data in these tables is numerical—nearly 70% [56], motivating the need for natural language interfaces that can perform data analysis via numerical question answering (QA).

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

In this paper, we study *Numerical Semi-Structured Table QA*, which aims to accurately compute numeric answer for the question over semi-structured tables. Its complexity stems from two aspects: *table side* and *question side*. On the table side, complexity mainly arises from *layout elements* that specify structural semantics for interpreting values. Specifically, a single table may contain multiple subtables where a cell's meaning is only recoverable by inferring its unique Hierarchical Header Sequence: (1.1) *hierarchical column headers*, where a column is specified using a nested column header sequence; (1.2) *hierarchical row headers*, where a row is specified using a nested row header sequence. We illustrate this using the table T shown in Figure 1:

> T contains two subtables: *Service Group Sale Info* and *Cloud Team Sale Info*. Within *Service Group Sale Info*:
> (1.1) Determining the year in *Year Ended May 31* requires inferring the column header sequence (*2016*, *Year Ended May 31*).
> (1.2) Determining the *Total Cost* requires inferring the row header sequence (*Managed Services*, *Total Cost*).

On the question side, complexity arises from: (2.1) *question uncertainty*, where questions are underspecified and require context-coherent grounding. (2.2) *Query intent*, where answering a question requires multi-step value grounding and reasoning, typically involving multiple *operators* and *operands*. As shown in Figure 1:

> (2.1): The question phrase "last two years" is underspecified and must be grounded to specific years (i.e., 2018 and 2017).
> (2.2): The query intent in "annualized uplift of total revenue in the year with the minimum cost" involves five steps: (1) locate data for the last two years, (2) rank these years by cost to find the minimum-cost year, (3) retrieve the values needed to instantiate the two operands for the uplift—the total revenue of that year and of the previous year, (4) aggregate the retrieved values, and (5) compute the uplift to produce the final answer.

Taken together, effectively addressing numerical semi-structured table QA requires a joint effort of **(C1)** *accurate table structure understanding*, **(C2)** *question uncertainty resolution*, and **(C3)** *complex query intent interpretation*. Existing approaches to semi-structured table QA typically fall into three families: *prompt-based* methods [52] that employ pretrained LLMs to directly generate answers, *code-driven* methods that translate questions into executable programs (e.g., SQL or Python), and *representation-driven* methods that convert tables into representations (e.g., trees or graphs) and then use LLM-guided retrieval and reasoning over the representations to answer questions. However, all three are rarely effective in handling the above multifaceted challenges in a holistic manner (see §2.2.1), as they simplify the table layout, or lack robust mechanisms for uncertainty resolution or complex query intent support. For instance, the most recent method ST-Raptor [40] first builds a hierarchical table tree and then uses LLMs to decompose a question

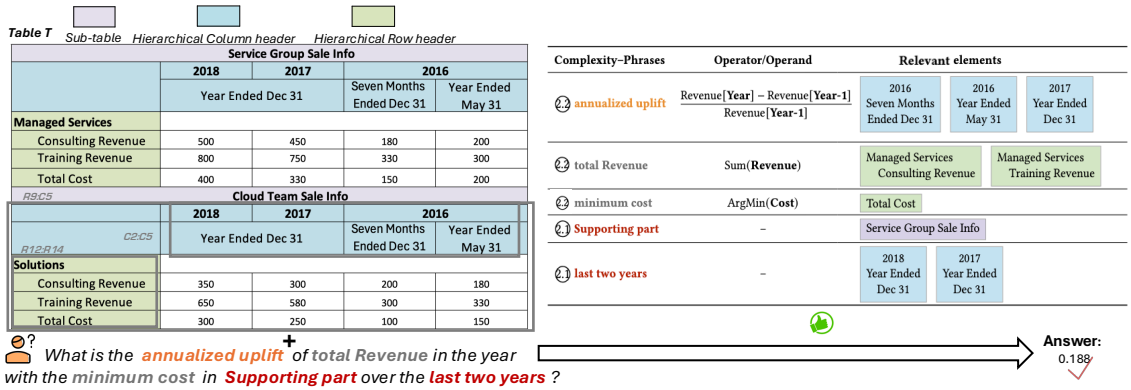


Figure 1: A running example illustrating our numerical semi-structured table QA problem.

into sub-questions and answer them over the tree. It still remains limited because: (1) its tree design misses key layout semantics (i.e., hierarchical row headers); (2) it does not explicitly resolve underspecified phrases, leading to imprecise decompositions that propagate to answering; (3) it answers each sub-question in isolation, without considering the step-by-step dependencies between sub-questions that often arise in complex query intents.

Beyond effectiveness, existing methods rely on high-capability LLMs to improve accuracy, but they ignore the significant monetary implications of large-scale deployments. A naive fix is to consider cheaper LLMs and route each instance (i.e., question and table pair) to an appropriate model to maximize overall accuracy under a fixed budget constraint. A typical LLM routing paradigm is *predict-then-optimize* [29, 30, 36], which learns per-instance answer quality and cost predictions for candidate LLMs using the training set (e.g., thousands of instances), and then solves a constraint optimization problem to derive the routing policy. However, this paradigm is impractical for real-world semi-structured table QA, where limited training set (e.g., tens of instances) [7] makes predictions unreliable [49]. The prediction errors then propagates through the optimization, yielding routing policies that neither reliably satisfy the budget nor achieve good answer accuracy. Consequently, we must address **C4:** *How to select an LLM for each instance, aiming to maximize accuracy under a fixed budget, given only limited training data?*

Our Contributions. We introduce SemiBonsai, a Cost-Effective, LLM-powered framework designed for Numerical Semi-Structured Table QA. SemiBonsai integrates a traceable pipeline with an routing mechanism, as shown in Figure 2.

• **Multiway Layered Table Structurer (§3).** To address **C1**, we propose a method that converts the table into a *Multiway Layered Tree* organized by structural elements, enabling more accurate table structure understanding.

• **Context-Aware Uncertainty Resolver (§4).** To address **C2**, we frame phrase grounding as an Extended Steiner Tree Problem over the Layered Tree, and use *context-aware pruning* to jointly select a minimal, context-coherent set of structural elements to ground underspecified phrases and rewrite the question.

• **Plan–Guided Reasoner (§5).** To address **C3**, we introduce the *Operand-based* and *Operator-based* decomposition strategies, to translate a complex query intent into a *query plan* organized by a set of interdependent subintents. Guided by this plan, it grounds

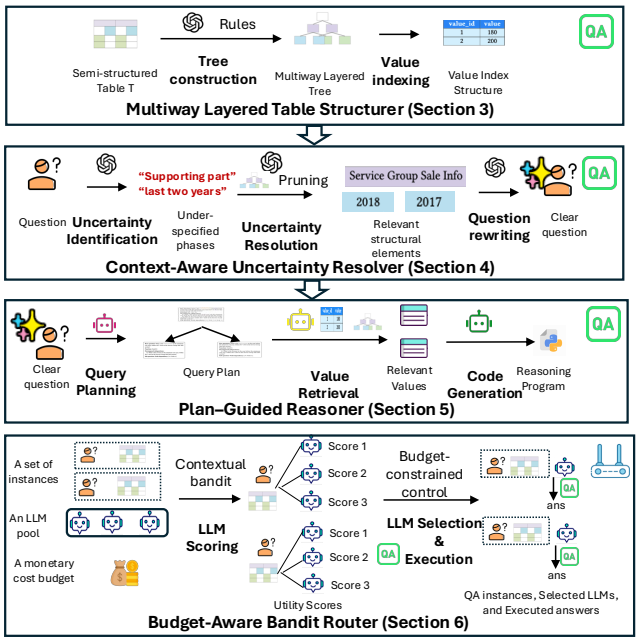


Figure 2: Overview of SemiBonsai, containing a traceable QA pipeline QA, and a cost-effective router.

each subintent to the required values and composes an executable reasoning program, enabling robust support for complex intents.

• **Budget-Aware Bandit Router (§6).** To address **C4**, we reformulate the budget-constrained routing objective into an *unconstrained Lagrangian surrogate objective*. Building on it, we propose a two-stage LLM routing strategy that (i) uses a contextual bandit algorithm to learn per-instance *utility scores* for candidate LLMs, and (ii) enforces the global budget constraint by selecting the LLM and adaptively updating the cost penalty during execution, which makes the routing effective even with limited training data.

• **Experimental Evaluation (§7).** Experiments on three benchmarks show that SemiBonsai improves both QA effectiveness and cost-effectiveness. Notably, the Table Structurer improves answer accuracy by 46%, and the Router achieves a 5% average relative gain in answer accuracy over existing routing approaches across monetary constraints.

Type	Method	Accurate table structure understanding				Question uncertainty resolution	Complex query intent interpretation
		Multiple subtables	Hierarchical column headers	Hierarchical row headers	Capability [2]		
Prompt-based [1]	TableLlama [52]	✗	✓	✓	⋆	⋆	⋆
Code-driven	TABFORMER [15]	✗	✓	✓	⋆	⋆	⋆
	E5 [55]	✗	✓	✓	⋆⋆	⋆⋆	⋆⋆
Representation-driven	ST-Raptor [40]	✓	✓	✗	⋆	⋆	⋆
	TreeThinker [45]	✗	✓	✓	⋆⋆	⋆⋆	⋆⋆
	GraphOTTER [25]	✗	✓	✓	⋆⋆	⋆⋆	⋆⋆
Hybrid [3]	SemiBonsai	✓	✓	✓	⋆ ⋆ ⋆	⋆ ⋆ ⋆	⋆ ⋆ ⋆

[1] We omit general-purpose foundation LLMs, as they are not tailored to semi-structured table QA.
[2] More ⋆ indicates stronger capability, based on the performance on our evaluation subsets requiring the corresponding capability (§7.2 - 7.3).
[3] We call it hybrid, as it contains table structure modeling (as in representation-driven methods) and reasoning program generation (as in code-driven methods).

Table 1: Comparison of semi-structured table QA methods based on the required capabilities.

2 PROBLEM FORMULATION AND LITERATURE REVIEW

2.1 Problem Definition

As commonly found in real-world data (e.g., financial reports), a semi-structured table T can be viewed as a 2D grid of cells arranged by rows and columns. A *column header* is a cell that labels a column, typically in the top rows. A *row header* is a cell that labels a row, typically in the left-most column. Layout cues such as indentation and merged cells are often used to indicate that one header is nested under another. We say two headers h_1 and h_2 satisfy a *nesting relation* $h_1 \triangleright h_2$ if the layout indicates that h_2 is nested under h_1. For example, in Figure 1, the column header *Year Ended May 31* is nested under the merged header *2016*, i.e., *2016*$\triangleright$ *Year Ended May 31*.

Hierarchical headers. A *hierarchical row header* is a maximal nested row header sequence $p^r = (h^r_1, \ldots, h^r_{L_1})$ of depth L_1 such that $h^r_i \triangleright h^r_{i+1}$ for all i, which identifies a row. A *hierarchical column header* is a maximal nested column header sequence $p^c = (h^c_1, \ldots, h^c_{L_2})$ of depth L_2 such that $h^c_i \triangleright h^c_{i+1}$ for all i, which identifies a column. If no nesting relations exist, we treat its depth as 1. As shown in Figure 1, (*Managed Services*, *Consulting Revenue*) is a hierarchical row header that identifies the 5th row, and (2018, *Year Ended Dec 31*) is a hierarchical column header that identifies the 2nd column.

Subtables. A *subtable* s can be viewed as a rectangular region in T delimited by a *vertically contiguous* group of hierarchical row headers $\mathcal{P}^r_s$ and a *horizontally contiguous* group of hierarchical column headers $\mathcal{P}^c_s$. It may further have an optional merged *title cell* whose text defines the subtable title $\mathcal{M}^t_s$, typically spanning all columns in $\mathcal{P}^c_s$ or all rows in $\mathcal{P}^r_s$. We define the subtable metadata as $\mathcal{M}_s = (\mathcal{M}^t_s, \mathcal{P}^r_s, \mathcal{P}^c_s)$. The content of s is denoted as $\mathcal{X}_s$, which is a set of cell values within the region delimited by $\mathcal{P}^r_s$ and $\mathcal{P}^c_s$.

For example, the grey boxes `C2:C5` and `R12:R14` in Figure 1 specify contiguous groups of hierarchical column headers and row headers, respectively, which together delimit the region `R9:C5` for a subtable titled *Cloud Team Sale Info*.

Semi-structured table. Accordingly, a semi-structured table is a collection of subtables within the 2D grid. Formally, we represent it as $T = \left(\mathcal{S}, \{\mathcal{M}_s, \mathcal{X}_s\}_{s \in \mathcal{S}}\right)$, where $\mathcal{S}$ is the set of subtables (($|\mathcal{S}| \geq 1$)). We refer to all subtable metadata as its structural elements: $\mathcal{E}(T) = \bigcup_{s \in \mathcal{S}} \mathcal{M}_s$.

For example, T in Figure 1 contains two subtables, distinguished by titles *Service Group Sale Info* and *Cloud Team Sale Info*. In this work, we assume that structural elements can be reliably indicated by layout cues; we do not consider cases where layout cues are misleading or relations can only be inferred using semantics across cell texts [6]. We focus on the common case where each subtable typically forms a rectangular region [27], and do not consider nested or interleaved subtables.

Definition 1 (Numerical Semi-Structured Table QA). *Given a semi-structured table T and a numerical question q (whose answer is a numeric value, obtained via arithmetic computation or aggregation), it aims to compute the correct answer, ans, by learning the function $f : (q, T) \rightarrow ans$.*

2.2 Literature Review

2.2.1 Semi-structured table QA. Table question answering aims to answer a natural language question using a table and has been studied extensively [26]. However, most prior work focuses on structured tables [23, 35, 43, 46, 50, 53] and does not fully account for semi-structured tables due to their complex layouts [54]. To bridge this gap, existing work for semi-structured table QA can be grouped into three categories (see Table 1).

Prompt-based methods, such as TableLlama [52] and GPT-5 [32], use instruction prompting [5] with foundation LLMs [32] or pre-trained tabular LLMs [52] to generate the answer to the question. Since both table structure and question understanding are handled implicitly by the LLM itself, these methods remain limited in answering numerical questions over the semi-structured table [38].

Code-driven methods, such as TABFORMER [15] and E5 [55], translate a natural-language question into an executable program (e.g., Python or SQL) and obtain an answer by executing it. TABFORMER [15] first normalizes a semi-structured table into a relational table and then generates SQL. While promising, it discards key structural elements (e.g., multiple subtables), leading to structural information loss that propagates to SQL generation. While E5

[55] improves QA robustness using LLMs by (i) producing structure summaries that capture hierarchy information and (ii) solving the question by generating a Python program, it lacks support for semi-structured tables with multiple subtables, as the LLM-generated summaries often fail to recognize subtable-level information (§7.5). Moreover, it relies on LLM to directly generate a reasoning program, without explicitly modeling uncertainty or complex intents.

Representation-driven methods, such as ST-Raptor [40], Tree-Thinker [45], and GraphOTTER [25], first convert the table into a representation (e.g., tree/graph) and then retrieve relevant values and reasoning over them guided by LLMs. ST-Raptor [40] constructs a hierarchical orthogonal tree using Visual Language Models (VLMs), then decomposes the question into sub-questions and answers them separately. TreeThinker [45] uses an LLM to infer hierarchical row/column header trees and then prompts LLMs to retrieve values and reason over them. GraphOTTER [25] models the table as an undirected graph linking neighboring headers/cells and answers by iteratively retrieving and expanding nodes before reasoning. However, these methods struggle on numerical QA over semi-structured tables: (1) their representations fail to capture all the structural elements (e.g., ST-Raptor omits hierarchical row headers), leading to incomplete structure understanding; (2) they rely on LLM to handle uncertainty and intents implicitly, lacking explicit mechanisms for uncertainty resolution and intent understanding.

2.2.2 LLM Routing. Similar to our objective in the Router to selecting an LLM for each instance, existing LLM routing studies—which route each input query to one LLM from a candidate pool—can be broadly categorized into three types based on their optimization objectives. *Quality-maximization routing* selects the model expected to yield the best answer quality for each query [9, 21, 47]. *Quality-constrained routing* aims to minimize the expected cost while ensuring the expected answer quality meets a required target over a query set [13, 29]. *Budget-constrained routing* maximizes expected answer quality while keeping the total cost within a given budget, which aligns with our cost-effective objective [33, 36, 44].

Budget-constrained routing methods typically follow a predict-then-optimize paradigm [36, 44]: they first learn predictors of per-instance answer quality and cost for each candidate LLM, then solve a constraint optimization problem to derive a routing policy. However, this paradigm is not applicable in our setting. A key limitation is that it assumes sufficiently large training data to obtain reliable instance-level predictions, while such training data is often limited in semi-structured table QA. Moreover, because both cost and accuracy can vary substantially across instances, it becomes even harder to learn accurate per-instance predictions. As a result, prediction errors can propagate into the optimization stage and degrade routing decisions. PILOT [33] can be a special case that trains a router online without requiring the training set, while it assumes online feedback of answer quality to update the routing decisions, which is unavailable in our setting.

3 MULTIWAY LAYERED TABLE STRUCTURER

Key issues. Existing methods interpret table structural semantics via one of the following ways:

• **W1:** Tree construction – building the hierarchy tree for the targeted table using an LLM/VLM [40, 45].

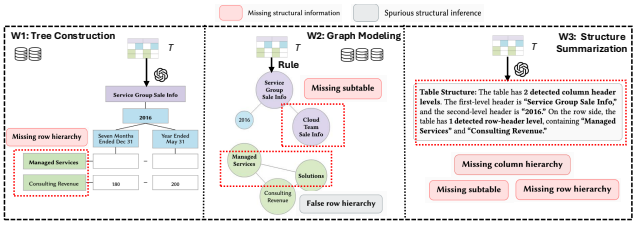


Figure 3: Error patterns in table structure understanding for representative strategies W1–W3. Required external storage is indicated by count; SemiBonsai uses one .

• **W2:** Graph modeling – connecting neighboring headers and cell values according to cell adjacency [25].

• **W3:** Structure summarization – prompting an LLM to output a table-level structural summary [55].

However, these approaches exhibit two failure modes: *missing structural information* (e.g., omitting header hierarchy) or *spurious structural inference* (e.g., hallucinating incorrect hierarchy). These errors arise from limited tree and graph designs *that cannot fully capture diverse structural elements* and from LLMs' *unreliable identification for each type of element.*

Example 1. *As shown in Figure 3, for the table T in Figure 4,* ***W1*** *misses the row hierarchy between* Managed Services *and* Consulting Revenue*;* ***W2*** *mixes the two subtables by linking* Service Group Sale Info *and* Cloud Team Sale Info *and hallucinates an incorrect row hierarchy (e.g.,* Managed Services–Solutions*);* ***W3*** *outputs a coarse summary that misses subtables and hierarchical row/column header.*

Moreover, we observe that existing tree/graph representations allocate a node per cell value without considering empty cells or duplicated values, thereby *incurring unnecessary storage overhead* for large table corpora with high sparsity and redundancy [4].

A novel structural model. To address them, we introduce a new structural model (Figure 4) consisting of: (1) a logical *Multiway Layered Tree*, which effectively captures diverse structural elements; (2) a physical *Value Index Structure*, which enables storage-efficient value lookup (§3.1). Given table T, we first construct the layered tree with a *rule-guided procedure* that exploits layout and styling cues to reliably capture diverse structural elements using a VLM (§3.2). We then use the tree to index table content by de-duplicating cell values and materializing the Value Index Structure (§3.3).

3.1 A Logical–Physical Structural Model

Concept 1 (Multiway Layered Tree). *Given a semi-structured table* $T = (\mathcal{S}, \{\mathcal{M}_s, \mathcal{X}_s\}_{s\in\mathcal{S}})$*, it is a rooted, directed, multiway layered tree* $G_T = (V, E)$ *that organizes subtables and hierarchical headers.*

Nodes. *V contains a unique root node and three types of nodes: (i) a subtable node for each $s \in \mathcal{S}$; (ii) a column header node for each header appearing in any hierarchical column header sequence in $\mathcal{P}^c_s$; and (iii) a row header node for each header appearing in any hierarchical row header sequence in $\mathcal{P}^r_s$.*

Edges. *E encodes parent–child relations layer by layer: (i) root→subtable edges; (ii) subtable→column header* and *subtable→row header edges that connect each subtable to the top-level headers of its hierarchical column and row headers; and (iii) column header→column header*

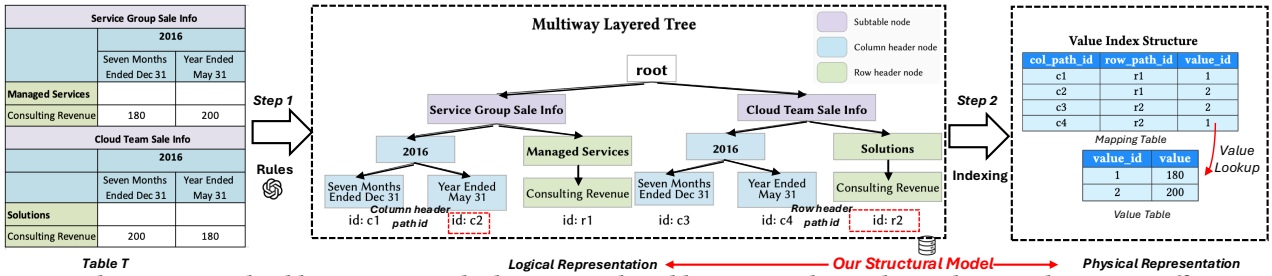


Figure 4: Multiway Layered Table Structurer, which converts the table into a multiway layered tree and a storage-efficient value index structure.

and *row header→row header edges that connect consecutive headers within each hierarchical header sequence.*

A path starting at the subtable and ending at a leaf column header/row header node uniquely identifies a hierarchical column/row header in a subtable. A row/column header subtree is a set of such paths sharing the same top-level row/column header node.

Example 2. As shown in Figure 4, id:c2 uniquely identifies a path that specifies a hierarchical column header of subtable Service Group Sale Info. *id:c1 and id:c2 belong to the same column header subtree.*

Concept 2 (Value Index Structure). *The value index structure consists of two relational tables: a Value Table and a Mapping Table.*

Value Table has two columns, value_id and value, where value_id is the primary key and value stores each distinct raw cell value once.

Mapping Table has three columns col_path_id, row_path_id, and value_id, where col_path_id and row_path_id identify root-to-leaf paths in G_T, and value_id refers to the corresponding cell value. Its primary key is the composite key (col_path_id, row_path_id), and its value_id is a foreign key referencing Value Table.value_id.

3.2 Constructing the layered tree

To construct a layered tree G_T, it requires identifying (i) the three types of nodes V, including subtable titles and headers; (ii) the parent–child edges E that capture *nesting relation* within hierarchical headers and the *containment relation* between each subtable and its headers. However, semi-structured tables do not specify such nodes and relations, and often express semantics via irregular layout cues (e.g., merged cells) and styling cues (e.g., boldface), which make reliable inference for semantics non-trivial. To address this, we introduce a *rule-guided two-stage procedure* to improve reliability: (1) use a VLM to identify candidate nodes, leveraging styling cues as additional signals, and (2) apply layout-driven rules to infer relations among candidates and construct a layered tree.

Candidate node identification. Considering that the subtable titles and headers can be distinguished by styling cues beyond plain text, we leverage a visual rendering of T and use a VLM to propose candidate nodes. Specifically, we use the HTML format of T, render it with a headless browser, and capture a high-resolution screenshot that preserves styling cues. We then prompt the VLM to propose candidate *subtable title*, *column header*, and *row header*, from each cell text, forming an initial node pool V.

Rule-guided tree construction. With the candidates, we can further prompt a VLM to infer nesting and containment relations from the rendered table image. However, this can be unreliable for large tables (e.g., > 20 rows) and complex layouts (e.g., > 10% merged cells). Because the number of candidates and possible relations grows quickly, making global relation inference error-prone and costly. Therefore, we construct G_T with a rule-guided procedure that scans the table and infers relations from layout cues, yielding a reliable Multiway Layered Tree, using three key steps:

Step 1: Identifying nesting relations and constructing header subtrees. Starting from the header candidates, we scan the table grid in a fixed order (top-to-bottom by rows and left-to-right by columns) and match candidates to concrete cells using n-gram similarity (above a threshold τ) to mitigate VLM hallucinations [40]. Matched cells are added to two confirmed pools for row and column headers, and are instantiated as row header or column header nodes accordingly. Meanwhile, we infer nesting relations and build edges by comparing each newly added header with previously confirmed headers in the same pool. Specifically, for row headers, when a new node h is added, we add edge from h_1 to h if (i) *indent rule*: h is indented and h_1 is the confirmed nearest preceding less-indented header, or (ii) *row-span rule*: h_1 is a confirmed merged cell whose row span covers h. For column headers, when a new node h is added, we add edge from h_1 to h if (iii) *col-span rule*: h_1 is a confirmed merged cell whose column span covers h. After linking the header nodes, we then build header subtrees by treating each top-level header (i.e., a node with no incoming edge) as a root and taking the set of all descendant header nodes reachable from it as the subtree.

Step 2: Identifying header subtree groups and inferring cross-adjacency. To identify the vertically/horizontally contiguous header groups that can delimit subtables, we record the rows/columns covered by each header subtree and merge subtrees into *maximal contiguous groups*. Specifically, for column header subtrees, we union those whose covered columns together form an inseparable horizontal interval; for row header subtrees, we union those whose covered rows have an inseparable vertical interval. Based on these header subtree groups, we identify their *cross-adjacency* via a *corner-adjacent rule*: a row header subtree group and a column header subtree group are *corner-adjacent* if the rightmost column covered by the row group is immediately left of the leftmost column covered by the column group, or the topmost row covered by the row group is immediately below the bottommost row covered by the column group.

Step 3: Identifying subtables via containment relations. We scan the grid and match title candidates to merged cells, instantiating each match as a title cell. For each title cell, we apply a *span-alignment rule*: if a title cell spans exactly the columns or rows covered by a

corner-adjacent pair of a column header subtree group and a row header subtree group, we create a subtable node and connect it to the root headers of the two subtree groups. Finally, we connect each subtable node to the root node, forming a layered tree.

3.3 Storage-efficient Value Indexing

Given the constructed layered tree G_T, we materialize the Value Index Structure by enumerating cell values indexed by structural elements and populating the Value Table and Mapping Table. Specifically, we traverse all structural elements via root-to-leaf column header paths and root-to-leaf row header paths of G_T. For each column and row header path pair that uniquely identifies a cell in the original table grid, we fetch the raw value of that cell and populate the two relational tables as follows. We first de-duplicate values in the Value Table: for each retrieved value not presented, we insert a tuple (value_id, value), otherwise we reuse the existing value_id. We then insert (col_path_id, row_path_id, value_id) into Mapping Table, recording the corresponding path pair and cell value.

4 CONTEXT-AWARE UNCERTAINTY RESOLVER

Key issues. Existing methods either lack question uncertainty resolution [15, 52], or handle it implicitly by independently mapping each phrase to structural elements[1] using LLMs [25, 45, 55] or embedding-based retrieval that assumes one-to-one mappings [40]. However, for questions with higher uncertainty (i.e., more under-specified phrases), these strategies often yield lower accuracy (§7.3) since they *miss one-to-many mappings and cannot jointly ground multiple phrases under a coherent context.*

Example 3. *As shown in Figure 1, the question contains two under-specified phrases: "Supporting part" and "last two years" Under a one-to-one assumption, it may miss that "last two years" corresponds to multiple headers (i.e.,* 2018 *and* 2017*). Moreover, resolving phrases independently can break their context coherence: the years for "last two years" should be grounded within the subtable-level context* Service Group Sale Info *identified by "Supporting part", while it retrieves* 2018 *and* 2017 *from an irrelevant subtable* Cloud Team Sale Info*, introducing spurious relevant elements and degrading accuracy.*

A context-aware pruning strategy. To address these issues, we propose a context-aware pruning strategy that ensures coherence by pruning irrelevant phrase groundings using the layered tree, supporting jointly one-to-many grounding (§4.1). Building on it, Context-Aware Uncertainty Resolver converts an uncertain question into a fully specified one (Figure 5): (1) uncertainty identification, (2) uncertainty resolution, and (3) question rewriting (§4.2).

4.1 Context-Aware Pruning

Intuitively, answering a question over a table requires identifying a set of structural elements that are not only relevant to its phrases, but also *context-coherent* with each other under the table's structure [24, 41], according to (i) *local context*, which constrains each phrase to structural elements structurally compatible (e.g., within the same subtable) [41]; (ii) *global context*, which prefers grounding

[1]Although these approaches support only limited structural element types, we assume the same set of structural elements for all to ensure fair comparison in this paper.

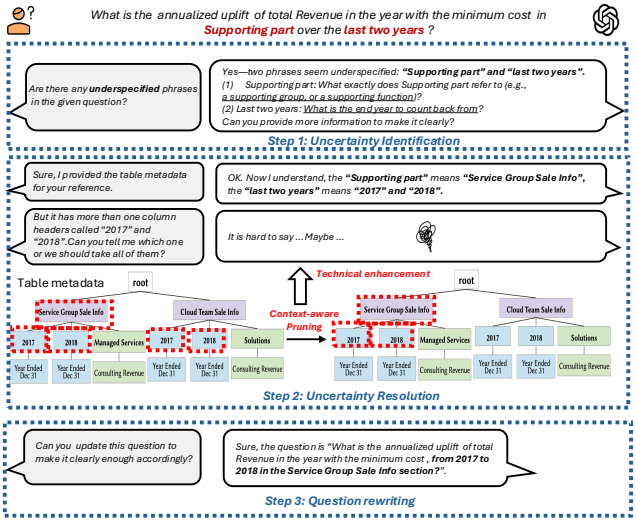


Figure 5: Context-Aware Uncertainty Resolver, which resolves the underspecified question into a fully specified one.

all phrases using as few such local contexts as possible, yielding a more coherent grounding [24]. Accordingly, we propose *context-aware pruning*, which aims to identify a minimal, context-coherent subset of structural elements that can jointly ground all phrases, thus pruning irrelevant candidates.

To achieve this, we model it as an extended Steiner Tree problem [22] over our layered tree: *given a candidate node set $C(u)$ for each under-specified phrase u, we aim to find a minimum-size collection of connected components of G_T such that, across the collection, every phrase is covered by at least one node.* As it is computationally intractable in general, we adopt the faster approximation algorithm [28] to obtain a solution. Then, for each obtained connected component, we enumerate its root-to-leaf subpaths and lift each subpath to the corresponding longest subtable-to-leaf header path in G_T that contains it. We then use the union of structural elements on these lifted paths as the pruned results.

Using this strategy, candidates not appearing on selected components are removed, as they are unsupported by local context (a connected component) and global context (a set of components).

4.2 From an Uncertain Question to a Clear One

Given an input question q with uncertainty, we need to resolve the uncertainty by converting it into a fully specified question $\tilde{q}$ via explicitly grounding each under-specified phrase to relevant structural elements using G_T.

Uncertainty identification. We call a phrase u in q *under-specified* if it cannot be resolved without grounding to the table, e.g., it is referentially unclear (e.g., shorthand), or has an unclear spatial/temporal scope (e.g., "last two years") [14, 51]. Such uncertainty makes the question underspecified without a table. We first prompt the LLM to extract under-specified phrases $\mathcal{U} = \{u_1, \ldots, u_{|\mathcal{U}|}\}$ from q.

Uncertainty resolution. For each under-specified phrase $u_i \in \mathcal{U}$, we first leverage an LLM to identify a candidate set $C(u_i)$ of nodes in G_T that may ground u_i (e.g., column header nodes "2017" and "2018" for "last two years"), considering such phrases often require

one-to-many groundings that go beyond embedding-based retrieval. To mitigate LLM hallucinations, we align each candidate to nodes in G_T using n-gram similarity with threshold τ, and discard unmatched candidates. We then apply *Context-Aware Pruning* strategy to obtain relevant structural elements based on the aligned candidates.

For large tables where G_T contains many nodes, which may reduce the effectiveness of LLM for candidate identification, we support the hybrid retrieval forming a small pool of tree nodes [50].

Question rewriting. Finally, we prompt an LLM to rewrite q into $\tilde{q}$ by explicitly grounding each u_i with the selected structural elements. For example, in Figure 5, given the selected subtable *Service Group Sale Info* and the hierarchical headers (*2017, Year Ended Dec 31*) and (*2018, Year Ended Dec 31*), the question is rewritten as: "What is the annualized uplift of total Revenue in the year with the minimum cost, from 2017 to 2018 in the Service Group Sale Info section?"

5 PLAN–GUIDED REASONER

Why do complex query intents break existing methods? A complex query intent generally involves multiple operators and operands. However, prompt-based approaches often fail to infer all operands for computation [38] and execute multiple operators [43]. Representation-driven methods are also brittle, as they either delegate the intent interpretation to the LLM or restrict it to a small set of predefined operators. Code-driven methods are more promising because they compile the question into an executable reasoning program (e.g., SQL/Python) that can support operators and operands for computation. However, they typically synthesize the full program in one shot, which becomes fragile when multiple operands and operators are involved [48].

Our plan-guided reasoning framework. To address this, we propose Plan–Guided Reasoner (Figure 6). It first uses a planning agent to generate a *Query Plan* by decomposing the complex query intent into subintents (§5.1). It then uses a grounding agent to ground each subintent with relevant values, and a coding agent to synthesize the executable reasoning program by following the subintent dependencies (§5.2). Guided by the query plan, it improves reasoning reliability for complex query intents.

5.1 Query Planning

Answering numerical questions with complex query intents often requires inferring multiple operands and composing multiple operators, making reasoning complicated. To simplify it, we *plan before solving*: we first translate the question into a query plan that decomposes the intent into a set of interdependent, *minimal subintents* that cannot be further decomposed, and specifies how they jointly resolve the intent, as guidance for solving.

What is the subintent? A *subintent* $\kappa = \langle \text{func}, \text{subqs}, \text{rawq}, \text{rel} \rangle$, specifying (1) which computation (i.e., a math expression) and operator (including aggregation, comparison, ranking, filtering, grouping) should be performed, using `Function` (func), (2) what table evidence is needed to supply its operands and parameters, using `Decomposed subquestions` (subqs), (3) `Raw question`, which is the natural language description of this subintent (rawq), and (4) optional `Dependency`, which reveals the dependencies between subintents (rel). For example, in Figure 6, Subintent 1 encodes the computation $(\#1 - \#2)/\#2$ and lists subquestions to supply operands

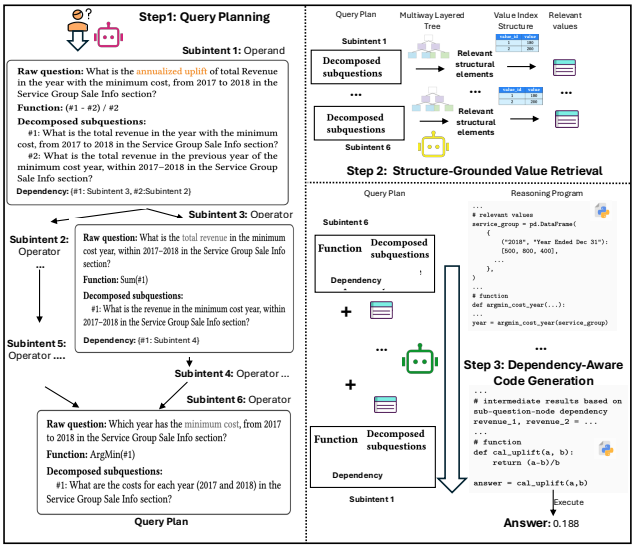


Figure 6: Plan–Guided Reasoner, which synthesizes an executable reasoning program using the planning, grounding, and coding agents.

#1 and #2; the dependency indicates that these operands are provided by Subintent 2 and Subintent 3.

Subintent-to-plan. Given the rewritten question $\tilde{q}$, we first initialize a root subintent κ_0 with $\tilde{q}$ as its rawq_0, and then use an LLM-based planning agent to perform decomposition based on $\tilde{q}$ by choosing one of two ways: (i) *Operand-based decomposition.* When $\tilde{q}$ involves a numerical computation, the agent first infers the computation as func_0 and then decomposes $\tilde{q}$ into a set of subquestions subqs_0, where each subquestion is designed to supply one distinct operand required by func_0; (ii) *Operator-based decomposition.* When $\tilde{q}$ involves a multi-operator chain, the agent extracts the first operator to apply as func_0, and rewrites $\tilde{q}$ into subqs_0 by splitting it into subquestions that produce the parameters for func_0.

After obtaining κ_0, we push it into a stack St and iteratively perform decomposition until St is empty. Specifically, at each step, we inspect the top Subintent $\kappa \in \text{St}$. We call κ *minimal* if none of its subquestions in subqs can be further decomposed by either operand-based or operator-based decomposition; in this case, we pop κ and add it to an Subintent set $\mathcal{K}$. Otherwise, for each decomposable subquestion in subqs, we apply operand-based or operator-based decomposition to obtain a new Subintent κ' similar to $\tilde{q}$. We then update κ's dependency rel to link the corresponding decomposable subquestion in κ to κ', and push κ' into St. Once St is empty, we construct the query plan by organizing $\mathcal{K}$ using the dependencies.

5.2 From Plan to Execution

With the produced query plan, we still need to derive the final answer, using two extra steps: (1) retrieve the relevant values as evidence for subqs in each Subintent, and (2) execute the subintents in dependency order to compose intermediate results progressively.

Structure-Grounded value retrieval. We traverse the query plan top-down and process each Subintent's subqs. For any independent subqs (i.e., it is not linked to another Subintent via rel), we retrieve

its required table evidence as follows. We first invoke an LLM-based grounding agent to extract key phrases and identify relevant nodes in the constructed layered tree for each phrase. We then apply the context-aware pruning strategy (§4.1) to select the most relevant structural elements. Finally, we use the selected elements' path identifiers to query the Value Index Structure: we look up the corresponding tuples in the Mapping Table and join their value_id to the Value Table to obtain the values. We return these values together with the elements as the grounded evidence for the subqs.

Dependency-Aware code generation. After obtaining all the required evidence, we generate an executable reasoning program by following the query plan bottom-up. We use Python (rather than SQL) for the reasoning program since our setting lacks an explicit database schema required for SQL generation. Specifically, we invoke an LLM-based coding agent to synthesize a subprogram for each Subintent: it (i) implements the Subintent's func and (ii) obtains func's operands/parameters either from retrieved evidence or from intermediate variables produced by dependent child subintents (as specified by rel). Finally, the agent composes these subprograms together following the dependency via rel and feeding each child's output variable into the corresponding operand/parameter of its parent, forming a reasoning program. If execution fails, we re-invoke coding agent with the error message to repair the code.

6 BUDGET-AWARE BANDIT ROUTER

Up to now, we have an LLM-powered QA pipeline that can produce an answer. However, relying on high-cost LLMs makes the pipeline impractical under budget constraints. Thereby, we incorporate smaller LLMs and study *budget-aware LLM routing for Numerical Semi-Structured Table QA* (§6.1), which enables budget-constrained deployments and turns our pipeline into a cost-effective solution. We then introduce our two-stage routing strategy (§6.2).

6.1 Problem Setup

Let $\mathcal{D} = \{(q_k, T_k)\}_{k=1}^{N}$ be a set of instances, where each instance consists of a numerical question q_k and a semi-structured table T_k. Let $\mathcal{L}$ be a pool of candidate LLMs. A routing policy is a mapping $\pi : \mathcal{D} \to \mathcal{L}$ that assigns one LLM to each instance.

Cost and answer accuracy. Given an LLM $\ell \in \mathcal{L}$, we instantiate our learned QA function f (Definition 1) with ℓ, denoted as $f_\ell : (q, T) \to \hat{ans}$. Applying f_ℓ on (q, T) also incurs a monetary cost $c(\ell, q, T) \in \mathbb{R}_{\geq 0}$. Answer accuracy is computed by comparing $\hat{ans}$ with the ground-truth answer y using a numerical equivalence function $\mathrm{Eq}(\cdot,\cdot)$, following [42]: $a(\ell, q, T) = \mathbf{1}\big[\mathrm{Eq}(\hat{ans}, y)\big] \in \{0, 1\}$.

Definition 2 (Budget-Aware LLM Routing for Numerical Semi-Structured Table QA). *Given query instances $\mathcal{D}$, an LLM candidate pool $\mathcal{L}$, and a total budget B, the goal is to learn a routing policy that selects one LLM per instance to maximize average answer accuracy under the budget constraint:*

$$\pi^\star = \arg\max_{\pi:\mathcal{D}\to\mathcal{L}} \frac{1}{N}\sum_{k=1}^{N} a(\pi(q_k, T_k),\, q_k,\, T_k) \tag{1}$$

$$\text{s.t.} \sum_{k=1}^{N} c(\pi(q_k, T_k),\, q_k,\, T_k) \;\leq\; B, \tag{2}$$

where $\pi(q_k, T_k)$ is the assigned LLM for the instance (q_k, T_k).

6.2 Two-Stage Routing: Contextual Bandit Scoring and Budget-Constrained Control

As discussed in §1, a limited training set makes the predict-then-optimize routing paradigm unreliable for our Budget-Aware LLM Routing problem , because it is hard to obtain accurate per-instance predictions of both answer accuracy and cost from only a few examples. A naive fix is to synthesize more training instances [7]. However, this is unlikely to work well for semi-structured table QA, as the complexity of answering each instance varies widely, making it hard to generate instances with similar complexity.

To address this, we apply a *Lagrangian relaxation* [17] to our budget-constrained routing objective (Appendix B [2]): we replace the global budget constraint with a cost penalty λ and obtain an *unconstrained Lagrangian surrogate objective* that scores each candidate LLM by the *utility* $u_\lambda(\ell; q, T) = a(\ell, q, T) - \lambda\, c(\ell, q, T)$. For a fixed λ, it then selects the LLM with the largest utility score u_λ, as a per-instance LLM selection rule induced by the relaxed objective. This avoids learning precise per-instance predictions of accuracy and cost ; instead, we directly learn to rank candidate LLMs by their utility for each instance, which is more robust with limited training data [12]. Since the Lagrangian relaxation replaces the hard budget constraint with a soft cost penalty, and may not enforce the budget, we therefore adapt λ to enforce the global budget constraint, building on these learned utilities. Accordingly, our routing follows two stages: (1) learn utility-based scoring for candidate LLMs; (2) enforce the budget by adaptively updating λ during execution.

LLM scoring. To effectively learn the instance-dependent utility scores, we use the contextual bandit algorithm [3], encoding each instance as a context vector x by concatenating the question with a flattened list of layered tree nodes for T and embedding the resulting text using a lightweight DistilBERT encoder [37]. Using the training set as offline bandit feedback—i.e., we only observe the utility of the LLM computed from each instance's true accuracy and cost—we train the contextual bandit to estimate the expected utility of each candidate LLM conditioned on x. At inference time, for a new instance (q, T), we compute x and output a utility estimate $\widehat{u}_\lambda(\ell; x)$ for every $\ell \in \mathcal{L}$, then route to the LLM with the largest estimated utility, $\ell^* = \arg\max_{\ell\in\mathcal{L}} \widehat{u}_\lambda(\ell; x)$.

Budget-Constrained control. To satisfy the global budget constraint B, given a sequence of instances $\{(q_k, T_k)\}_{k=1}^{N}$, we maintain the cumulative cost C_{k-1} and a cost-penalty parameter λ_k before routing the k-th instance. For the current instance, we score each candidate $\ell \in \mathcal{L}$ as $\widehat{u}_{\lambda_k}(\ell; x_k)$ with the current penalty λ_k, and select the LLM with the highest utility score. After executing the chosen LLM and observing its cost c_k, we update $C_k = C_{k-1} + c_k$ and adjust the penalty via a dual update $\lambda_{k+1} \leftarrow \big[\lambda_k + \eta\cdot(c_k - \frac{B}{N})\big]_+$, where $\eta > 0$ is a step size and $[\cdot]_+$ clamps negative values to zero.

Rationale behind our framework. SemiBonsai starts with the Table Structurer to interpret and cache the table's structural semantics, making them reusable for all questions over the same table. It then applies the Uncertainty Resolver to ground under-specified phrases and produce a clear question, which is necessary for the Reasoner to reliably translate the complex intents into a query plan. Finally, since these stages are LLM-powered, it places a cost-effective Router on top of them to select LLMs under a budget.

7 EXPERIMENTAL EVALUATION

QA effectiveness, robustness, and efficiency. We assess the effectiveness, robustness, and efficiency of SemiBonsai for numerical QA over semi-structured tables by answering the following questions:

- Q1: How effective is SemiBonsai on numerical semi-structured table QA overall?
- Q2: How robust is SemiBonsai to varying table side complexity and question side complexity?
- Q3: How efficient is SemiBonsai for online QA in terms of runtime and token usage?
- Q4: What is the contribution of each core QA component in SemiBonsai to QA effectiveness, and how does each component contribute to the overall time efficiency and token efficiency?

Cost-effectiveness. We assess the cost-effectiveness of SemiBonsai:

- Q5: Can the Budget-Aware Bandit Router in SemiBonsai keep the total monetary cost within a given budget while maximizing the answer accuracy?

7.1 Experimental Setup

7.1.1 Datasets. We evaluate on three widely used semi-structured table QA benchmarks [15, 45]: HiTab [11], MultiHiertt [56], and RealHiTBench [45]. We choose them because their tables have much higher numerical value ratios, i.e., the fraction of cells with numerical values (0.72/0.68/0.79), which provides more numerical questions that match our setting.[2] While other semi-structured table QA benchmarks are more text-dominated, with a much lower numerical value ratio. For example, TempTabQA [19] has 0.04, InfoTabs [20] has 0.03, WikiTQ [34] has 0.26, and SSTQA [40] has 0.40, making them less suitable for evaluation.

Using these selected benchmarks, we derive numerical-only subsets by using numerical questions that can be answered from a single table in the HiTab and MultiHiertt validation splits, and all numerical questions in RealHiTBench. We denote the resulting datasets as HiTab-Num, MultiHiertt-Num, and RealHiTBench-Num.

Complexity of Questions and Tables. As discussed in §1, the QA effectiveness depends on the complexity of questions and tables. To assess this, we curate three subsets using the above three datasets: Complex-Q (complex-question subset), Complex-T (complex-table subset), and Complex-QT (both complex question and complex table), by automatically classifying questions and tables into simple or complex. A straightforward approach is to define a heuristic complexity score using the relevant factors and apply a fixed threshold. However, the score is sensitive to how multiple factors (e.g., the number and nesting depths of structural elements) interact, and a single threshold may misclassify borderline cases. Therefore, we train feature-based classifiers to learn a data-driven decision boundary that accounts for multiple factors. Specifically, we (i) construct table-side and question-side complexity features, (ii) build small labeled pools manually, and (iii) train two binary classifiers (for table-side and question-side complexity), which are then applied to all benchmarks to form Complex-Q, Complex-T, and Complex-QT. Details are provided in Appendix §A [2].

Table 2 summarizes key factors that contribute to table-side and question-side complexity. For tables, we measure (i) the number of subtables, (ii) the mean nesting depth of hierarchical column headers, and (iii) the mean nesting depth of hierarchical row headers. For questions, we measure (i) question uncertainty via the under-specified ratio (the fraction of key phrases not directly matched with annotated relevant structural elements), (ii) the number of grounded operands, and (iii) the number of required operators.

[2]In addition, these tables are generally small (i.e., fewer than 100 rows).

Dataset	#T	Table			#Q	Question		
		#Sub	Avg. ColDepth	Avg. RowDepth		Avg. UnderSpec	#Operands	#Operators
HiTab-Num	80	1.00	2.20	1.51	203	0.51	2.07	3.26
MultiHiertt-Num	158	1.00	1.61	1.56	240	0.66	2.61	3.56
RealHiTBench-Num	191	1.34	2.00	1.51	276	0.31	2.52	3.18
Complex-Q	201	1.11	1.89	1.50	295	**0.70**	**2.81**	**3.76**
Complex-T	130	**1.14**	**2.25**	**2.01**	206	0.53	2.46	3.30
Complex-QT	83	**1.13**	**2.17**	**1.99**	107	**0.70**	**2.69**	**3.43**

#Sub: #subtables; ColDepth/RowDepth: nesting depth; UnderSpec: under-specified ratio.

Table 2: Dataset statistics.

7.1.2 Baselines. We compare SemiBonsai with representative semi-structured table QA baselines from three categories. Prompt-based methods using GPT-5 [32] and TableLlama [52]. Code-driven methods include E5 [55] and TABFORMER [15]. Since there is no publicly available implementation for TABFORMER, we reimplement its pipeline by adopting NormTab [31] for table normalization and using an LLM-based SQL generator. Representation-driven methods include ST-Raptor [40], TreeThinker [45], and GraphOTTER [25]. The details of each method can be found in §2.2.

7.1.3 Evaluation metrics. To measure answer accuracy on numerical questions, we adopt Arithmetic Exact Match (EM) following [42], which measures the percentage of predictions whose numeric values match the ground truth. Since the prediction and gold answer can be formatted differently yet remain equivalent (e.g., 10% vs. 0.1), we apply an LLM-based value normalization and equivalence check for comparison following [40], enabling a more nuanced evaluation.

7.1.4 Implementation. We use GPT-4.1 as the primary backend LLM for all baselines and SemiBonsai. For the table structurer, we use GPT-4.1 as the VLM. The matching threshold τ is set to 0.9. Since methods support different input table formats, we provide each table in two equivalent formats (HTML and Excel). We use HTML for prompt-based and code-driven methods, and Excel for SemiBonsai to align with representation-driven methods. The prompts used by SemiBonsai are included in Appendix §D [2]. Router implementation details of SemiBonsai are provided in §7.6.

7.2 QA Effectiveness (Q1)

In this subsection, we first report overall QA effectiveness on HiTab-Num, MultiHiertt-Num, and RealHiTBench-Num. We then evaluate the complex subsets Complex-Q, Complex-T, and Complex-QT.

7.2.1 Main results. Table 3 summarizes answer accuracy on three semi-structured table QA benchmarks. From the results, we observe: (1) SemiBonsai significantly outperforms all existing semi-structured QA baselines, achieving about an 18% average improvement over the second-best baseline in each benchmark. This highlights the effectiveness of SemiBonsai for semi-structured table QA, as it reliably models table structures, question uncertainty, and complex query intents—capabilities that prior methods do not

Method	HiTab-Num	MultiHierrtt-Num	RealHiTBench-Num
TableLlama	0.333	0.096	0.079
GPT-5	0.603	0.338	0.502
TABFORMER	0.123	0.225	0.220
E5	0.422	<u>0.588</u>	0.627
ST-Raptor	0.319	0.354	0.308
TreeThinker	0.554	0.221	0.519
GraphOTTER	<u>0.618</u>	0.279	<u>0.631</u>
SemiBonsai	**0.724**	**0.708**	**0.744**

Table 3: Answer accuracy across three benchmarks using EM.

Method	Complex-Q	Complex-T	Complex-QT
TableLlama	0.095	0.131	0.075
GPT-5	0.341	0.434	0.389
TABFORMER	0.121	0.127	0.093
E5	<u>0.430</u>	<u>0.514</u>	<u>0.481</u>
ST-Raptor	0.224	0.255	0.176
TreeThinker	0.301	0.401	0.287
GraphOTTER	0.388	0.467	0.389
SemiBonsai	**0.646**$^{(+50\%)}$	**0.693**$^{(+35\%)}$	**0.714**$^{(+48\%)}$

Table 4: Answer accuracy using EM on the three subsets. SemiBonsai also reports relative improvement over the best baseline (underline).

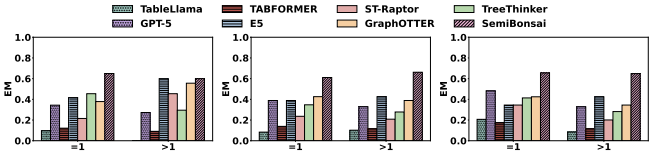
address explicitly or effectively. (2) No single baseline is consistently strong across all benchmarks, because they prioritize different capabilities. Representation-driven methods better capture table structures (e.g., GraphOTTER on HiTab-Num/RealHiTBench-Num), while code-driven methods better handle complex intents and question uncertainty (e.g., E5 on MultiHiertt-Num).

7.2.2 Results on complex questions and tables. Table 4 reports accuracy on three complex subsets. We observe that: (1) On Complex-Q, SemiBonsai yields a 50% improvement over the best baseline, suggesting that explicitly resolving question-side complexity is critical, where existing methods often fail. (2) On Complex-T, SemiBonsai also achieves clear gains, showing that effective table-structure modeling is necessary for complex tables. (3) Overall, SemiBonsai consistently attains the highest accuracy across all subsets, indicating its robust performance for both complex tables and questions.

7.3 Robustness (Q2)

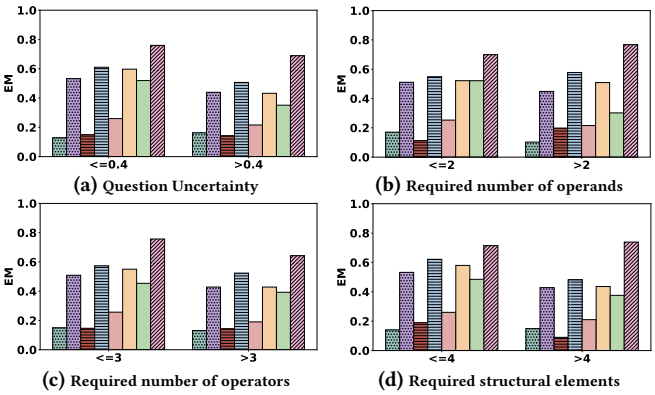
In this subsection, we analyze robustness with respect to fine-grained table-side and question-side factors that can increase complexity and affect QA effectiveness. To focus on these complex cases while controlling the other side, we evaluate table-side factors on Complex-Q and question-side factors on Complex-T. In addition to the factors in Table 2, we also include *value grounding complexity* on the question side, since questions with uncertainty and complex query intents typically require grounding to a larger set of structural elements to locate all necessary values; we approximate it by the number of required structural elements.

[3]Some results may be visible because the corresponding values are 0.



(a) Subtables. (b) Column header depths. (c) Row header depths.

Figure 7: Robustness under table-side factors on Complex-Q.[3]



(a) Question Uncertainty (b) Required number of operands (c) Required number of operators (d) Required structural elements

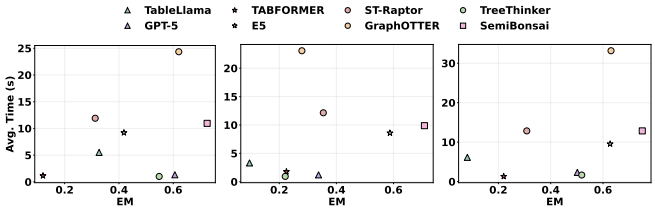
Figure 8: Robustness under question-side factors on Complex-T.

7.3.1 Impact of table-side factors. Since table-side factors are discrete, we use rule-based splits: single vs. multiple subtables, and hierarchical headers when the mean nesting depth exceeds 1. Figure 7 shows that (1) most baselines degrade noticeably as table-side complexity increases, because they cannot reliably capture diverse structural elements, which downgrade answering accuracy. (2) SemiBonsai is the most robust across all splits, because its Layered Tree offers a more reliable table structure, enabling more accurate value grounding even with multiple subtables and deeper header hierarchies. Considering existing benchmarks contain few extremely complex tables (e.g., header depths > 3), we build a complexity-augmented subset by reusing Complex-Q. Results are in Appendix §C [2].

7.3.2 Impact of question-side factors. Since question-side factors are continuous and more dispersed, we split the subset into low vs. high groups using the median of its empirical distribution. Figure 8 shows that: (1) Most baselines degrade from low to high across all factors, especially for question uncertainty and value grounding complexity (Figure 8a and Figure 8d), because these settings make it harder to identify the relevant structural elements. (2) SemiBonsai remains the most robust, achieving the highest accuracy in both groups with smaller drops as question-side complexity increases, demonstrating its effectiveness in handling question uncertainty and complex intents, and identifying relevant structural elements.

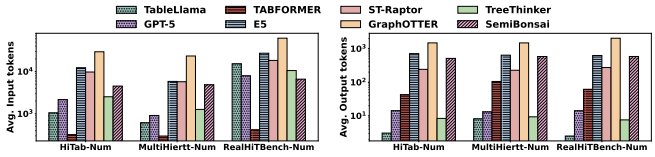
7.4 QA Efficiency (Q3)

In this subsection, we first compare time efficiency by measuring the per-instance latency to produce an answer. For representation-based methods and SemiBonsai, we do not include representation construction time as it is offline. Since all baselines are LLM-based,



(a) HiTab-Num. (b) MultiHiertt-Num. (c) RealHiTBench-Num.

Figure 9: Per-instance runtime, shown together with answer accuracy to illustrate the trade-off.



(a) Input tokens per instance (b) Output tokens per instance

Figure 10: Input and output token usage comparison.

we compare token efficiency by reporting input and output token usage[4]. A component-level breakdown of SemiBonsai is in §7.5.

7.4.1 Time efficiency. Figure 9 illustrates the results. We observe that: (1) Prompt-based methods are typically the fastest, as they answer with a single LLM inference. GraphOTTER is the slowest because it repeatedly invokes LLMs to traverse its graph and retrieve relevant values before producing an answer. Compared with these baselines, SemiBonsai incurs a modest runtime. (2) Across all benchmarks, SemiBonsai offers the best accuracy–latency trade-off, achieving the highest EM while remaining competitive in runtime.

7.4.2 Token efficiency. Figure 10a compares the input token usage. Prompt-based methods use the fewest tokens because their input is mainly the question plus a linearized table; accordingly, their input grows on RealHiTBench-Num, whose tables are larger (avg. 35 rows and 14 columns). In contrast, other baselines rely primarily on layout information and show more varied input-token usage: TABFORMER is minimal since it only feeds the normalized schema and question, while the remaining methods repeatedly invoke LLMs for intermediate reasoning, incurring much higher input tokens. SemiBonsai is moderate. Figure 10b compares the output token usage. Prompt-based methods and TreeThinker use the fewest tokens because each call only needs the LLM to output a single value, whereas other methods consume more tokens since they rely on longer intermediate LLM reasoning outputs to derive the answer.

7.5 Ablation and Component Analysis (Q4)

In this subsection, we study component-level QA effectiveness and efficiency. First, we conduct an ablation study over the three core QA components in SemiBonsai to quantify their contribution to overall answer accuracy. Second, we report a per-component breakdown of runtime and token usage, highlighting their efficiency impact.

7.5.1 Effectiveness. Since the three QA components are executed sequentially and removing any one breaks the end-to-end QA, we instead ablate the key designs within each component while keeping the remaining pipeline unchanged.

[4]Since TableLlama uses its own tokenizer, we report its token usage under the GPT-4.1 tokenizer for consistency.

Variant	HiTab-Num		MultiHiertt-Num		RealHiTBench-Num		
	Row[1]	Col[1]	Row	Col	Subtable[2]	Row	Col
LLM Tree	0.830	0.716	0.783	0.746	–[3]	0.488	0.404
VLM Tree	0.823	0.822	0.792	0.869	0.649	0.467	0.646
SemiBonsai	**0.903**	**0.824**	**0.898**	**0.927**	**0.816**	**0.677**	**0.716**

[1] "Row"/"Col" denote hierarchical row/column headers.
[2] Only RealHiTBench-Num contains multiple subtables.
[3] LLM Tree does not produce subtable titles on RealHiTBench-Num.

Table 5: Layered tree reliability using F1.

Variant	HiTab-Num	MultiHiertt-Num	RealHiTBench-Num
LLM Tree	$0.444^{\downarrow 39\%}$	$0.326^{\downarrow 54\%}$	$0.413^{\downarrow 45\%}$
VLM Tree	$0.438^{\downarrow 40\%}$	$0.294^{\downarrow 59\%}$	$0.457^{\downarrow 39\%}$
GT Tree	0.846	0.783	0.888
No-Resolver	$0.572^{\downarrow 21\%}$	$0.648^{\downarrow 9\%}$	$0.735^{\downarrow 1\%}$
No-Pruning	$0.639^{\downarrow 12\%}$	$0.690^{\downarrow 3\%}$	$0.743^{\downarrow 0\%}$
No-Plan	$0.706^{\downarrow 3\%}$	$0.690^{\downarrow 3\%}$	$0.743^{\downarrow 0\%}$
SemiBonsai	**0.724**	**0.708**	**0.744**

Table 6: Ablation study using EM.

Structurer. Since it aims to produce (i) a reliable logical Layered Tree for answering and (ii) a storage-efficient physical Value Index Structure for value lookup, we study:

(1) Layered tree reliability and QA effectiveness. To validate the benefit of our rule-guided Layered Tree construction, we compare the following variants: LLM Tree (LLM-only construction), VLM Tree (VLM-only construction from the rendered table image), and GT Tree (Layered Tree built from ground-truth structural elements).

We evaluate Layered Tree reliability by matching the extracted structural elements against annotated ground truth and reporting F1 for each type following [18]. We treat a predicted structural element as a match only under: the full ordered header sequence and the label text exactly match the ground truth. Table 5 shows that SemiBonsai achieves the highest F1 across all element types and datasets, indicating that our rule-guided construction produces more reliable Layered Trees than pure LLM/VLM.

We further report the answer accuracy when using each tree in the same pipeline. Table 6 shows that SemiBonsai achieves substantially higher accuracy than LLM Tree and VLM Tree, because its rule-guided construction yields a more reliable Layered Tree, which directly improves QA effectiveness. SemiBonsai also performs comparably to GT Tree; the remaining gap mainly comes from errors in VLM-based candidate node identification.

(2) Value index struture storage cost. To validate the benefit of our value index structure, we compare with Raw Index (storing one value per cell without de-duplication). On HiTab-Num, MultiHiertt-Num, and RealHiTBench-Num, Raw Index requires 0.9/1.2/5.4 MB, while our Value Index Structure requires only 0.2/0.4/0.8 MB, yielding an average storage reduction of 75%.

Resolver. To quantify the effectiveness of Context-Aware Uncertainty Resolver, we compare answer accuracy under: No-Pruning (disable our context-aware pruning), and No-Resolver (remove the entire uncertainty resolver module). Table 6 shows that explicitly

Module	HiTab-Num			MultiHiertt-Num			RealHiTBench-Num		
	Time	InTok[1]	OutTok	Time	InTok	OutTok	Time	InTok	OutTok
Structurer	14.2	3,969	236	14.0	3,912	176	13.2	3,747	329
Resolver	2.7	736	138	1.9	702	133	1.4	1,215	46
Reasoner	8.3	3,802	375	8.0	4,144	450	11.4	5,404	538
Planning	3.8	2,500	137	4.1	2,815	173	4.9	3,449	161
Grounding	1.8	538	92	1.6	509	108	2.4	843	145
Coding	2.7	764	146	2.3	820	170	4.1	1,112	232

[1] InTok/OutTok: number of input/output tokens.

Table 7: Per-module efficiency of SemiBonsai: runtime (seconds) and token usage per instance.

resolving question uncertainty via the Resolver improves answer accuracy, and that context-aware pruning contributes to these gains via generating candidate groundings and pruning them into a coherent context. Note that improvement on RealHiTBench-Num is smaller, due to its lower uncertainty (0.31).

Reasoner. To assess the impact of our query plan on QA effectiveness, we compare with: No-Plan (disable query planning). Table 6 shows that incorporating the plan improves accuracy by providing explicit guidance for reasoning. Again, the gain on RealHiTBench-Num is smaller, as it contains fewer questions with complex intents.

7.5.2 Efficiency. For Table Structurer, which preprocesses each table into our structural model based on a VLM, we report per-table preprocessing time and token usage. For Uncertainty Resolver and Reasoner, which operate online, we report per-question runtime and token usage for end-to-end QA. As Reasoner incorporates planning, grounding, and coding agents, we further provide each breakdown.

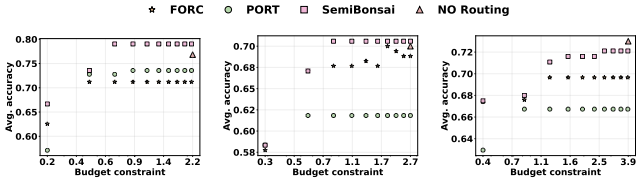
The results are shown in Table 7. The Structurer costs the most because it invokes the VLM to propose nodes from the table image, then scans the full table to confirm the candidate nodes, and finally infers the structural relations. Although this may be expensive, it is a one-time cost cached over all questions on the same table. For online QA, the Resolver is lightweight since it only grounds uncertain phrases and rewrites the question, while the Reasoner dominates usage due to query planning by decomposing the intents.

7.6 Cost-effectiveness (Q5)

7.6.1 Experimental setup. To evaluate the cost-effectiveness of SemiBonsai, we compare against two representative budget-aware LLM routing baselines [5], FORC [36] and PORT [44]. FORC trains a meta-model to predict instance-level accuracy and cost, and then applies ILP to optimize routing decisions. PORT estimates per-instance answer accuracy and cost by averaging the retrieved similar instances, and then uses an online optimization algorithm for routing. We also report NO Routing, which always uses the default large LLM, as a reference point on answer accuracy.

Evaluation instances, LLM pool and budget constraints. We evaluate on the three semi-structured table QA benchmarks. Since our goal is to compare routing methods under a limited training set, we mimic such a setting and ensure enough instances for evaluation: for each benchmark, we randomly sample 20% of instances for training and use the remaining 80% for evaluation. We use an LLM pool consisting of the default LLM GPT-4.1 and six smaller LLMs, including GPT–4/5 mini variants and Gemini 2.5-flash variants [1]. We construct 10 budget constraints by linearly spacing between the minimum and maximum total costs of using a single LLM.

Implementation. For SemiBonsai, we normalize costs to be on the same scale as accuracy, set the contextual bandit exploration parameter to 0.8 with ridge regularization 1.0; for the dual update, the initialize cost penalty is 0 and η is 1. For the baselines, we use default hyperparameters and set PORT's retrieval size as 5 given the small training set. Since these baselines may exceed the budget, we evaluate all methods under a budget-stop protocol: we execute selections sequentially and stop before the next selection would violate the total budget, and report the accuracy over the evaluation set. Considering ordering sensitivity, we repeat each constraint for 5 runs with different permutations and report average.



(a) HiTab-Num. **(b)** MultiHiertt-Num. **(c)** RealHiTBench-Num.

Figure 11: Cost–effectiveness: average accuracy achieved under different budget constraints.

7.6.2 Main Results. Figure 11 reports the average accuracy achieved by different monetary budget constraints. Observing: (1) Across all three benchmarks and budgets, SemiBonsai attains the highest accuracy, compared with routing baselines. This highlights the effectiveness of our Budget-Aware Bandit Router, which learns per-instance utility for each LLM and adapts selections with budget control, making it more robust with limited training data under the constraint. (2) As the budget constraint becomes looser, SemiBonsai steadily increases accuracy and remains stable. This indicates it can effectively trade cost for answer accuracy, by adaptively updating the cost penalty and route to stronger LLMs when the budget allows. (3) By leveraging smaller LLMs, SemiBonsai can even surpass NO Routing (see Figure 11a and 11b). This suggests that using routing can not only reduce the cost, but also improve answer accuracy by selecting a better-performing model for each instance.

8 CONCLUSION

In this paper, we propose SemiBonsai, a cost-effective, LLM-powered framework designed for numerical QA over semi-structured tables. SemiBonsai consists of: Multiway Layered Table Structurer, which converts a table into a multiway layered tree; Context-Aware Uncertainty Resolver, which grounds underspecified phrases to context-coherent structural elements and rewrites the question; Plan–Guided Reasoner, which decomposes complex intent into a query plan and generates an executable reasoning program; Budget-Aware Bandit Router, which learns per-instance routing utilities and applies a budget-constrained control to maximize accuracy under a fixed budget. Experiments on three benchmarks show that SemiBonsai consistently improves answer accuracy, and achieves strong cost-effectiveness.

[5] While prior work such as FrugalGPT [8] could be adapted to handle budget constraints, it focuses on LLM ensembling—invoking multiple LLMs for a single instance—which differs from our single routing objective; we therefore omit it from comparison.

REFERENCES

[1] [n. d.]. Gemini 2.5 Flash. https://cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/

[2] [n. d.]. Technical Report. https://github.com/JrJessyLuo/SemiBonsai/blob/main/technical_report.pdf.

[3] Alina Beygelzimer, John Langford, Lihong Li, Lev Reyzin, and Robert Schapire. 2011. Contextual bandit algorithms with supervised learning guarantees. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*. JMLR Workshop and Conference Proceedings, 19–26.

[4] Jan-Micha Bodensohn, Ulf Brackmann, Liane Vogel, Anupam Sanghi, and Carsten Binnig. 2025. Unveiling Challenges for LLMs in Enterprise Data Engineering. *Proc. VLDB Endow.* 19, 2 (2025), 196–209.

[5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[6] Bin Cao, Huixian Lu, Chenwen Ma, Ting Wang, Ruizhe Li, and Jing Fan. 2026. Orthogonal Hierarchical Decomposition for Structure-Aware Table Understanding with Large Language Models. *arXiv preprint arXiv:2602.01969* (2026).

[7] Chengliang Chai, Jiajun Li, Yuhao Deng, Yuanhao Zhong, Ye Yuan, Guoren Wang, and Lei Cao. 2025. Doctopus: Budget-aware structural table extraction from unstructured documents. *Proceedings of the VLDB Endowment* 18, 11 (2025), 3695–3707.

[8] Lingjiao Chen, Matei Zaharia, and James Zou. 2024. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Trans. Mach. Learn. Res.* 2024 (2024).

[9] Shuhao Chen, Weisen Jiang, Baijiong Lin, James Kwok, and Yu Zhang. 2024. Routerdc: Query-based router by dual contrastive learning for assembling large language models. *Advances in Neural Information Processing Systems* 37 (2024), 66305–66328.

[10] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. *CoRR* abs/1603.02754 (2016).

[11] Zhoujun Cheng, Haoyu Dong, Zhiruo Wang, Ran Jia, Jiaqi Guo, Yan Gao, Shi Han, Jian-Guang Lou, and Dongmei Zhang. 2022. HiTab: A Hierarchical Table Dataset for Question Answering and Natural Language Generation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1094–1110.

[12] Maria Dimakopoulou, Zhimei Ren, and Zhengyuan Zhou. 2021. Online multi-armed bandits with adaptive inference. *Advances in Neural Information Processing Systems* 34 (2021), 1939–1951.

[13] Dujian Ding, Ankur Mallick, Shaokun Zhang, Chi Wang, Daniel Madrigal, Mirian del Carmen Hipolito Garcia, Menglin Xia, Laks V. S. Lakshmanan, Qingyun Wu, and Victor Rühle. 2025. BEST-Route: Adaptive LLM Routing with Test-Time Optimal Compute. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*. OpenReview.net.

[14] Zhongjun Ding, Yin Lin, and Tianjing Zeng. 2025. Ambisql: Interactive ambiguity detection and resolution for text-to-sql. *arXiv preprint arXiv:2508.15276* (2025).

[15] Haoyu Dong, Yue Hu, and Yanan Cao. 2025. Reasoning and Retrieval for Complex Semi-structured Tables via Reinforced Relational Data Transformation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1382–1391.

[16] Joakim Edin, Alexander Junge, Jakob D Havtorn, Lasse Borgholt, Maria Maistro, Tuukka Ruotsalo, and Lars Maaløe. 2023. Automated medical coding on MIMIC-III and MIMIC-IV: a critical review and replicability study. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2572–2582.

[17] Marshall L Fisher. 1981. The Lagrangian relaxation method for solving integer programming problems. *Management science* 27, 1 (1981), 1–18.

[18] Ling Fu, Zhebin Kuang, Jiajun Song, Mingxin Huang, Biao Yang, Yuzhe Li, Linghao Zhu, Qidi Luo, Xinyu Wang, Hao Lu, et al. [n. d.]. OCRBench v2: An Improved Benchmark for Evaluating Large Multimodal Models on Visual Text Localization and Reasoning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

[19] Vivek Gupta, Pranshu Kandoi, Mahek Vora, Shuo Zhang, Yujie He, Ridho Reinanda, and Vivek Srikumar. 2023. TempTabQA: Temporal question answering for semi-structured tables. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2431–2453.

[20] Vivek Gupta, Maitrey Mehta, Pegah Nokhiz, and Vivek Srikumar. 2020. INFOTABS: Inference on tables as semi-structured data. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 2309–2324.

[21] Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. [n. d.]. RouterBench: A Benchmark for Multi-LLM Routing System. In *Agentic Markets Workshop at ICML 2024*.

[22] Frank K Hwang and Dana S Richards. 1992. Steiner tree problems. *Networks* 22, 1 (1992), 55–89.

[23] Yangfan Jiang, Fei Wei, Ergute Bao, Yaliang Li, Bolin Ding, Yin Yang, and Xiaokui Xiao. 2026. Accurate Table Question Answering with Accessible LLMs. *arXiv preprint arXiv:2601.03137* (2026).

[24] Mayank Kothyari, Dhruva Dhingra, Sunita Sarawagi, and Soumen Chakrabarti. 2023. CRUSH4SQL: Collective retrieval using schema hallucination for Text2SQL. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 14054–14066.

[25] Qianlong Li, Chen Huang, Shuai Li, Yuanxin Xiang, Deng Xiong, and Wenqiang Lei. 2025. GraphOTTER: Evolving LLM-based Graph Reasoning for Complex Table Question Answering. In *Proceedings of the 31st International Conference on Computational Linguistics*. 5486–5506.

[26] Weizheng Lu, Jing Zhang, Ju Fan, Zihao Fu, Yueguo Chen, and Xiaoyong Du. 2025. Large language model for table processing: A survey. *Frontiers of Computer Science* 19, 2 (2025), 192350.

[27] Kexin Huang Jan Eric Lenssen Rishabh Ranjan Joshua Robinson Rex Ying Jiaxuan You Jure Leskovec Matthias Fey, Weihua Hu. 2023. Relational Deep Learning: Graph Representation Learning on Relational Tables. (2023).

[28] Kurt Mehlhorn. 1988. A faster approximation algorithm for the Steiner problem in graphs. *Inform. Process. Lett.* 27, 3 (1988), 125–128.

[29] Kai Mei, Wujiang Xu, Minghao Guo, Shuhang Lin, and Yongfeng Zhang. 2025. Omnirouter: Budget and performance controllable multi-llm routing. *arXiv preprint arXiv:2502.20576* (2025).

[30] Alireza Mohammadshahi, Arshad Rafiq Shaikh, and Majid Yazdani. 2024. Routoo: Learning to route to large language models effectively. *arXiv preprint arXiv:2401.13979* (2024).

[31] Md Nahid and Davood Rafiei. 2024. NormTab: Improving Symbolic Reasoning in LLMs Through Tabular Data Normalization. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 3569–3585.

[32] OpenAI. 2026. GPT-5. https://platform.openai.com/docs/models/gpt-5.

[33] Pranoy Panda, Raghav Magazine, Chaitanya Devaguptapu, Sho Takemori, and Vishal Sharma. 2025. Adaptive LLM Routing under Budget Constraints. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. 23934–23949.

[34] Panupong Pasupat and Percy Liang. 2015. Compositional semantic parsing on semi-structured tables. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. 1470–1480.

[35] Ivan A Poddubnyi and Nikita O Dorodnykh. 2025. Query Plan Generation for Table Question Answering. *Proceedings of the VLDB Endowment. ISSN* 2150 (2025), 8097.

[36] Marija Šakota, Maxime Peyrard, and Robert West. 2024. Fly-swat or cannon? cost-effective language model choice via meta-modeling. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 606–615.

[37] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108* (2019).

[38] Eli Schwartz, Leshem Choshen, Joseph Shtok, Sivan Doveh, Leonid Karlinsky, and Assaf Arbelle. 2024. Numerologic: Number encoding for enhanced llms' numerical reasoning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 206–212.

[39] Pragya Srivastava, Manuj Malik, Vivek Gupta, Tanuja Ganu, and Dan Roth. 2024. Evaluating LLMs' Mathematical Reasoning in Financial Document Question Answering. In *Findings of the Association for Computational Linguistics ACL 2024*. 3853–3878.

[40] Zirui Tang, Boyu Niu, Xuanhe Zhou, Boxiu Li, Wei Zhou, Jiannan Wang, Guoliang Li, Xinyi Zhang, and Fan Wu. 2025. ST-Raptor: LLM-Powered Semi-Structured Table Question Answering. *Proc. ACM Manag. Data* 3, 6 (2025), 1–27.

[41] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. 2020. Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers. In *Proceedings of the 58th annual meeting of the association for computational linguistics*. 7567–7578.

[42] Dingzirui Wang, Longxu Dou, Xuanliang Zhang, Qingfu Zhu, and Wanxiang Che. 2024. Enhancing Numerical Reasoning with the Guidance of Reliable Reasoning Processes. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 10812–10828.

[43] Zilong Wang, Hao Zhang, Chun-Liang Li, Julian Martin Eisenschlos, Vincent Perot, Zifeng Wang, Lesly Miculicich, Yasuhisa Fujii, Jingbo Shang, Chen-Yu Lee, et al. [n. d.]. Chain-of-Table: Evolving Tables in the Reasoning Chain for Table Understanding. In *The Twelfth International Conference on Learning Representations*.

[44] Fangzhou Wu and Sandeep Silwal. 2025. PORT: Efficient Training-Free Online Routing for High-Volume Multi-LLM Serving. In *NeurIPS 2025 Workshop on Machine Learning for Systems (MLForSys)*.

[45] Pengzuo Wu, Yuhang Yang, Guangcheng Zhu, Chao Ye, Hong Gu, Xu Lu, Ruixuan Xiao, Bowen Bao, Yijing He, Liangyu Zha, et al. 2025. Realhitbench: A comprehensive realistic hierarchical table benchmark for evaluating llm-based table analysis. In *Findings of the Association for Computational Linguistics: ACL 2025*. 7105–7137.

[46] Guorui Xiao, Dong He, Jin Wang, and Magdalena Balazinska. 2025. Cents: A Flexible and Cost-Effective Framework for LLM-Based Table Understanding.

Proceedings of the VLDB Endowment 18, 11 (2025), 4574–4587.

[47] Cheng Yan, Wuyang Zhang, Zhiyuan Ning, Fan Xu, Ziyang Tao, Lu Zhang, Bing Yin, and Yanyong Zhang. 2026. Breaking Model Lock-in: Cost-Efficient Zero-Shot LLM Routing via a Universal Latent Space. *arXiv preprint arXiv:2601.06220* (2026).

[48] Dayu Yang, Tianyang Liu, Daoan Zhang, Antoine Simoulin, Xiaoyi Liu, Yuwei Cao, Zhaopu Teng, Xin Qian, Grey Yang, Jiebo Luo, et al. 2025. Code to think, think to code: A survey on code-enhanced reasoning and reasoning-driven code intelligence in llms. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 2586–2616.

[49] Sepanta Zeighami, Shreya Shankar, and Aditya Parameswaran. 2025. Cut Costs, Not Accuracy: LLM-Powered Data Processing with Guarantees. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–26.

[50] Chi Zhang, Meihui Zhang, Yuxin Yang, Tao Chen, and Zhaojing Luo. 2025. AixelAsk: A Stepwise-Guided Retrieval and Reasoning Framework for Large Table QA. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–25.

[51] Meng Zhang, Kexin Ma, Liyang Xu, Kedi Zhang, Yuanxi Peng, and Ruochun Jin. 2025. CLEAR: A Parser-Independent Disambiguation Framework for NL2SQL. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 1–14.

[52] Tianshu Zhang, Xiang Yue, Yifei Li, and Huan Sun. 2024. Tablellama: Towards open large generalist models for tables. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 6024–6044.

[53] Yunjia Zhang, Jordan Henkel, Avrilia Floratou, Joyce Cahoon, Shaleen Deep, and Jignesh M Patel. 2024. ReAcTable: Enhancing ReAct for Table Question Answering. *Proceedings of the VLDB Endowment* 17, 8 (2024), 1981–1994.

[54] Yue Zhang, Seiji Maekawa, and Nikita Bhutani. 2025. Same Content, Different Representations: A Controlled Study for Table QA. *arXiv preprint arXiv:2509.22983* (2025).

[55] Zhehao Zhang, Yan Gao, and Jian-Guang Lou. 2024. e5: Zero-shot hierarchical table analysis using augmented LLMs via explain, extract, execute, exhibit and extrapolate. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 1244–1258.

[56] Yilun Zhao, Yunxiang Li, Chenying Li, and Rui Zhang. 2022. MULTIHIERTT: Numerical Reasoning over Multi Hierarchical Tabular and Textual Data. In *60th Annual Meeting of the Association for Computational Linguistics, ACL 2022*. Association for Computational Linguistics (ACL), 6588–6600.

A COMPLEXITY OF QUESTIONS AND TABLES

As discussed in §1, the effectiveness depends on the complexity of questions and tables. To assess this, we curate three subsets using the above three datasets: Complex-Q (complex-question subset), Complex-T (complex-table subset), and Complex-QT (both complex question and complex table), by automatically classifying questions and tables into simple or complex. A straightforward approach is to define a heuristic complexity score using the relevant factors and apply a fixed threshold. However, the score is sensitive to how multiple factors (e.g., the number and nesting depths of structural elements) interact, and a single threshold may misclassify borderline cases. Therefore, we train feature-based classifiers to learn a data-driven decision boundary that accounts for multiple factors. Specifically, we (i) construct table-side and question-side complexity features, (ii) build small labeled pools manually, and (iii) train two binary classifiers (for table-side and question-side complexity), which are then applied to all benchmarks to form Complex-Q, Complex-T, and Complex-QT.

A.1 Feature Construction

Table-side features. We extract table-side features using the annotated structural elements provided by each benchmark. Specifically, we compute: (i) the number of subtables, (ii) the number of hierarchical row headers and column headers, and (iii) the statistics (min/mean/max) of their nested depths.

Question-side features. We characterize question complexity based on: (i) *Question uncertainty.* We compute an under-specified ratio, defined as the fraction of key phrases not exactly match the annotated structural elements. Since only HiTab-Num provides key phrase annotations, for the other datasets, we use an LLM to extract key phrases grounded on the table structural elements and then compute the ratio. (ii) *Complex query intent.* We compute the number of operators and operands, using the annotated reasoning programs and steps. Since each operator or operand may require traversing multiple structural elements to locate the required values, introducing additional complexity, we compute the number of involved subtables, hierarchical row headers, and hierarchical column headers, for the question, along with their nesting statistics (min/mean/max).

A.2 Complexity Subset Construction

Constructing labeled pools. We build two small labeled pools (simple vs. complex) for tables (69/56) and questions (139/142) separately as follows.

Labeling tables. We define a heuristic table complexity score as a weighted sum of two core Table-side features following [40]: (a) the number of subtables and (b) the mean nesting level of hierarchical row/column headers, with each feature z-score normalized. We rank tables by this score and take the top 20% as complex and the bottom 20% as simple. We then manually filter obvious false positives/negatives (e.g., cases where nearly all QA baselines succeed on "complex" candidates, or fail on "simple" candidates).

Labeling questions. Similarly, we define a heuristic question complexity score as a weighted sum of two core Question-side features: (a) the underspecified ratio and (b) the total number of operands and operators, again with z-score normalization. We rank questions by this score, take the top/bottom 20% as complex/simple candidates, and apply the same manual filtering.

Training classifiers and obtaining subsets. Using the labeled pools and all constructed features, we train two XGBoost [10] binary classifiers—one to predict whether a table is complex and the other to predict whether a question is complex. Leveraging richer features allows the classifiers to capture diverse complexity factors beyond the heuristic score and yields more reliable classification results. We then apply the classifiers to all examples in the three datasets to obtain complex questions and tables.

B LAGRANGIAN RELAXATION FOR ROUTING

Lagrangian relaxation. We start from the budget-aware routing objective (Definition 2), which can be written as the constrained maximization:

$$\max_{\pi:\mathcal{D}\to\mathcal{L}} \quad \frac{1}{N}\sum_{k=1}^{N} a(\pi(q_k,T_k),\, q_k,\, T_k) \tag{3}$$

$$\text{s.t.} \quad \sum_{k=1}^{N} c(\pi(q_k,T_k),\, q_k,\, T_k) \le B. \tag{4}$$

Because the policy π makes a discrete choice for each instance, directly solving (3)–(4) is generally hard. We therefore apply a *Lagrangian relaxation* [17] by introducing a nonnegative multiplier $\lambda \ge 0$ for the budget constraint and forming the Lagrangian:

$$\begin{aligned}\mathcal{L}(\pi,\lambda) &= \frac{1}{N}\sum_{k=1}^{N} a(\pi(q_k,T_k),\, q_k,\, T_k) - \lambda\left(\sum_{k=1}^{N} c(\pi(q_k,T_k),\, q_k,\, T_k) - B\right)\\ &= \lambda B \;+\; \sum_{k=1}^{N}\left(\frac{1}{N}a(\pi(q_k,T_k),\, q_k,\, T_k) - \lambda\, c(\pi(q_k,T_k),\, q_k,\, T_k)\right).\end{aligned} \tag{5}$$

For a fixed λ, the term λB is constant with respect to π. Hence maximizing $\mathcal{L}(\pi,\lambda)$ over π decomposes across instances, yielding an *instance-wise* decision rule. Specifically, for each instance (q_k, T_k), the optimal choice under the relaxed objective is

$$\pi_\lambda(q_k,T_k) \in \arg\max_{\ell\in\mathcal{L}}\left(\frac{1}{N}a(\ell,q_k,T_k) - \lambda\, c(\ell,q_k,T_k)\right). \tag{6}$$

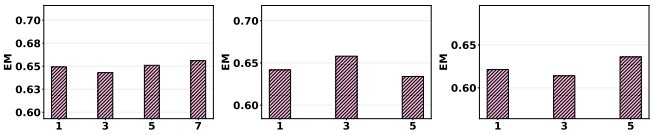
Unconstrained Lagrangian Surrogate Objective. Since multiplying the objective by a positive constant does not change the arg max, we can absorb the factor $1/N$ into the penalty weight by defining $\tilde{\lambda} = \lambda N$. This yields the per-instance *utility*:

$$\pi_{\tilde{\lambda}}(q_k,T_k) \in \arg\max_{\ell\in\mathcal{L}} \underbrace{\left(a(\ell,q_k,T_k) - \tilde{\lambda}\, c(\ell,q_k,T_k)\right)}_{u_{\tilde{\lambda}}(\ell;q_k,T_k)}. \tag{7}$$

C ROBUSTNESS UNDER COMPLEXITY-AUGMENTED TABLES

Considering that existing benchmarks contain few extremely complex tables (e.g., header depths >3), we build a complexity-augmented subset by reusing the original QA instances in Complex-Q and only increasing table-side complexity while keeping the question–answer grounding unchanged.

Specifically, for each factor, we first select the lowest-complexity tables (e.g., with a single subtable or depth =1), then prompt an LLM



(a) Subtable. **(b)** Column header depth. **(c)** Row header depth.

Figure 12: Robustness of SemiBonsai under table-side augmentation on Complex-Q.

to reorganize the same table content to meet a target complexity level (e.g., increasing header nesting depth or adding more subtables) based on the observed structural elements, without altering the values. We control complexity with discrete targets: subtables {3,5,7} and row/column header depth {3,5}, chosen to cover complex settings while matching the observed upper bounds in the benchmarks (i.e., max subtables is 7 and depth is 5).

This produces three complexity-augmented collections (subtable, row-header, and column-header augmentation), where each original QA instance is paired with multiple synthesized table variants at different complexity levels. At each target level, we have 381 questions with 192 subtable-augmented table variants, 228 questions with 103 row-header–augmented variants, and 82 questions with 52 column-header–augmented variants.

Results. As shown in Figure 12, SemiBonsai remains stable under table-side augmentation as we increase the number of subtables and deepen row/column header hierarchies. This indicates that its structural model is robust and continues to support reliable downstream answering.

D PROMPTS

The following lists all the prompts in SemiBonsai, including the candidate node identification prompt (§3.2) of Multiway Layered Table Structurer, uncertainty identification and question rewriting prompts (§4.2) of Context-Aware Uncertainty Resolver, planning/-grounding/coding prompts (§5) of Plan–Guided Reasoner.

Candidate Node Identification Prompt

The table - understanding agent is required to extract the table's header structure from a TABLE IMAGE and return a JSON object in a specific format.

- JSON Format:
  - `column_header_groups`: Represent column header structure, can be multi - level or flat.
  - `row_header_groups`: Represent row header structure, can be multi - level or flat.
- Column Headers:
  - Identify leaf headers (aligned with data columns, left - to - right).
  - Handle spanning/merged headers: Different output formats based on position relative to leaf headers.
  - Output flat items if no spanning/shared headers.
  - Do not mix flat and spanning - based items for the same columns.
  - Use only cell texts.
- Row Headers:
  - Check left - most column below header band.
- Rules:
  - Do not mix columns from different visual blocks.
  - Use verbatim text from the image.
  - Output only the specified JSON object.

Uncertainty Identification Prompt

You are a table-question grounding assistant.
You are given:

- (1) a question, and
- (2) table metadata node labels (unique labels), consisting of:
  - `subtable_titles`: section/subtable titles
  - `column_headers`: column header node labels
  - `row_headers`: row header node labels

Your task has two parts:

- A) Identify underspecified query phrases
  - Extract all phrases in the question that are underspecified/ambiguous in the context of the table metadata.
  - A phrase is underspecified if it cannot be grounded to a specific table element without interpretation, *or* it could refer to multiple metadata nodes.
  - If there is no underspecified phrase, return `{"phrase_groundings": []}`.
- B) Ground each phrase to relevant metadata node labels (one-to-many)
  - For each underspecified phrase, select all relevant node labels it could refer to.
  - Return selections in three buckets: `subtable_titles` / `column_headers` / `row_headers`.
  - Selections must be strictly chosen from the provided metadata lists (no invented labels).
  - Rank the selected labels within each bucket by confidence (most likely first).
  - If a bucket has no relevant labels, return an empty list for that bucket.
- Strict rules
  - Use only labels that appear in the provided metadata lists (verbatim; no paraphrasing).
  - Do not do any global consistency or cross-phrase pruning (handled later).
  - Output must be valid JSON and nothing else.
- Output JSON format

```
{
  "phrase_groundings": [
    {
      "phrase": "...",
      "selected_nodes": {
        "subtable_titles": [...],
        "column_headers": [...],
        "row_headers": [...]
      }
    }
  ]
}
```

Question: {question}
Metadata: {table_metadata}

Question Rewriting Prompt

You are given:

- (1) an original question, and
- (2) a set of selected structural elements that map underspecified phrases.

Your task:

- Rewrite the question into a clear, fully specified version that explicitly mentions the grounded labels.
- Preserve the original intent and semantics.
- Do not add any new facts not implied by the groundings.

Rules

- Use the grounded labels verbatim.
- Output must be valid JSON and nothing else.

Output JSON format

```
{
  "rewritten_question": "...",
  "used_groundings": [
    {
      "phrase": "...",
      "structural_elements": [...]
    }
  ]
}
```

Original Question: {question}
Selected elements: {selected_groundings}

Planning Prompt

You are an LLM-based planning agent for numerical table QA.
Subintent. $\kappa = \langle$`func`, `subqs`, `rawq`, `rel`$\rangle$: `func` (operator/computation), `subqs` (subquestions for operands/params), `rawq` (NL), `rel` (dependencies).
Goal. Pick one unresolved target from `subquestions` using `action history`, then output the next action.
Allowed actions (pick one). `infer_calculation_formula` `multihop_question_decomposition` `generate_execute_program`
Rules. (i) For the first two actions, the first argument must be the target subquestion ID. (ii) If decomposing, create new unused subquestion IDs.
Decision.

- **Directly executable** (retrieve cell(s) from metadata, optionally simple aggregation; no formula/hops) → `generate_execute_program`.
- **Numerical computation** (operands decomposition) → `infer_calculation_formula`: infer minimal `func` using operators in {`operator_set`} with placeholders #1,#2,...; create `subqs` where each new subquestion supplies one operand; record dependencies in `rel`.
- **Multi-hop reasoning** (operators decomposition) → `multihop_question_decomposition`: extract the first operator as `func`, decompose into ≤ 3 single-hop subquestions (ordered); encode inter-hop dependencies in `rel`.

Inputs. `subquestions: {subquestions}` `table_metadata: {table_metadata}` `action_history: {operation_history}` `possible_actions: {possible_actions}`
Output (JSON only).

```
{"function":"...","explanation":"..."}
```

Grounding Prompt

You are given a question and table metadata labels: `subtable_titles`, `column_headers`, `row_headers`.
Task. (1) Extract key query phrases. (2) For each phrase, select all relevant labels from the metadata (one-to-many), grouped into the three buckets above and ranked by confidence.
Rules. Use only provided labels (verbatim). Output valid JSON only.
Output (JSON).

```
{"phrase_groundings":[{"phrase":"...","selected_nodes":{
  "subtable_titles":[...],"column_headers":[...],"row_headers":[...]}}]}
```

Question: {question} **Metadata:** {table_metadata}

Coding Prompt

Generate one complete, directly executable Python snippet to answer the raw numerical question using the given atomic subquestions and relevant subtable data.
Critical output rules. (i) Output only a single ```python ... ``` block. (ii) No comments in code. (iii) Assign the final result to `final_answer`.
Data rules (Pandas). Build a minimal DataFrame via `pd.DataFrame` from a Python dict using only the relevant subtable; include only needed rows/columns; normalize numeric strings (e.g., commas, %, currency) to `int`/`float`; filter out summary rows (e.g., "Total", "Subtotal") if they affect aggregation.
Task. Compute intermediate values for atomic subquestions first, then combine them to answer the raw question. Use reasoning hints and program evidence if helpful.
Inputs. `atomic_subquestions: {atomic_subquestions}` `relevant_subtable: {atomic_subquestions_subdata}` `subquestions: {subquestions}` `reasoning_history: {reasoning_history}` `program_evidence: {numerical_reasoning_context}`